Jogo da Forca
Cliente–Servidor sobre TCP/IP

Rafael Gomes nº51923, Leonardo Ferreira nº52981, Tomás Simões nº52585

***Resumo*—Este relatório descreve a implementação de um jogo da forca multijogador em Java, desenvolvido segundo uma arquitetura cliente–servidor sobre TCP/IP. O trabalho centra-se na ligação entre clientes e servidor, na manutenção do estado do jogo no servidor e no protocolo textual usado para trocar mensagens. O sistema aceita vários clientes, atribui identificadores, inicia o jogo quando as condições definidas são cumpridas, atualiza o estado durante a partida, trata tempos limite e encerra as ligações no fim do jogo. O relatório descreve ainda a organização do código e as opções tomadas na implementação.**

***Index Terms*—Java, TCP/IP, sockets, cliente–servidor, jogo em rede, sistemas distribuídos.**

I. Introdução

O trabalho proposto para a unidade curricular de Sistemas Distribuídos tem como objetivo o desenvolvimento de um jogo da forca multijogador suportado por uma arquitetura cliente–servidor. Nesta abordagem, a lógica do sistema fica concentrada num servidor central, enquanto os clientes mostram o estado do jogo ao utilizador e permitem enviar jogadas.

Este modelo permite trabalhar conceitos centrais de sistemas distribuídos, como comunicação por sockets, sincronização de eventos, gestão de concorrência e manutenção de um estado partilhado coerente. Em vez de cada participante executar uma cópia autónoma do jogo, existe um único estado de jogo no servidor. É o servidor que aceita novas ligações, atribui identificadores, escolhe a palavra, valida jogadas e determina o momento em que a partida termina.

O projeto está organizado em três blocos principais: servidor, cliente e componentes partilhados. Cada cliente mantém uma ligação TCP aberta com o servidor durante toda a partida. Essa ligação é usada para enviar mensagens de controlo, atualizações de estado e jogadas.

II. Objetivos do Trabalho

Do ponto de vista funcional, o projeto disponibiliza uma partida de forca com dois a quatro participantes, uma palavra comum escolhida aleatoriamente e um conjunto fixo de tentativas partilhadas. Do ponto de vista técnico, o objetivo é demonstrar uma solução distribuída baseada em TCP/IP, com comunicação simples e tratamento de entradas inválidas, desistências e falta de resposta dentro do tempo limite.

O jogo foi estruturado em turnos rotativos, com um jogador ativo em cada ronda.

Licenciatura em Engenharia Informática, Universidade da Beira Interior. Unidade curricular: Sistemas Distribuídos.

III. Arquitetura e Organização do Projeto

O código-fonte encontra-se organizado em três pacotes: `client`, `server` e `shared`. Esta separação ajuda a distinguir responsabilidades e a evitar duplicação de informação.

No lado do servidor existem três classes com funções complementares. A classe `Server` é o ponto de entrada da aplicação e gere o ciclo do jogo: abertura do porto, aceitação de clientes, arranque do lobby, início da partida e encerramento do servidor. A classe `ClientHandler` representa cada ligação estabelecida com um cliente, encapsulando o respetivo `Socket` e os canais de entrada e saída. Por fim, a classe `GameManager` mantém o estado do jogo, incluindo a palavra escolhida, a máscara atual, as tentativas restantes, o conjunto de letras já usadas e a identificação do vencedor.

No lado do cliente existe a classe `Client`, responsável por abrir a ligação ao servidor, tratar as mensagens recebidas e recolher a entrada do utilizador na consola. O pacote `shared` contém a classe `Protocol`, onde estão definidos os nomes das mensagens e os parâmetros principais do sistema, e a classe `WordList`, que guarda a lista fixa de 100 palavras.

A lógica do jogo é mantida no servidor, enquanto o cliente apenas envia comandos. Isto reduz o risco de inconsistências entre participantes e simplifica a validação das regras.

IV. Ligação Cliente–Servidor

A. Estabelecimento da ligação

A comunicação assenta nas classes `ServerSocket` e `Socket` da biblioteca padrão de Java [1], [2]. O servidor escuta no porto definido em `Protocol.PORT`, atualmente configurado para `8080`. Sempre que um novo cliente tenta ligar-se, o servidor executa `accept()`, cria um novo objeto `ClientHandler` e atribui-lhe um identificador.

Depois de criado o `Socket`, o cliente e o servidor estabelecem dois fluxos de comunicação complementares: um fluxo de entrada, baseado em `BufferedReader`, para leitura linha a linha; e um fluxo de saída, baseado em `PrintWriter`, para envio imediato das mensagens. Como cada mensagem corresponde a uma linha de texto, o protocolo é simples de testar e de analisar.

Assim que a ligação fica pronta, o servidor envia ao cliente uma mensagem de boas-vindas com o formato `WELCOME <id> <players_total>`. Esta primeira mensagem tem duas funções: informar o utilizador do número que lhe foi atribuído no jogo e indicar quantos jogadores estão ligados naquele instante.

B. Manutenção da sessão

Após a fase inicial de ligação, a sessão mantém-se ativa até ao fim da partida ou até ocorrer uma desconexão. O servidor guarda uma lista de jogadores ligados, usando uma `CopyOnWriteArrayList`, e remove periodicamente os elementos cujo socket já não esteja disponível. Esta limpeza é possível porque o `ServerSocket` usa um tempo limite curto de aceitação, o que impede o ciclo principal de ficar bloqueado indefinidamente.

Cada cliente mantém uma ligação persistente ao longo de toda a sessão. Não existe reconexão automática nem abertura de um novo socket por jogada. Cada participante entra uma vez, recebe o seu contexto de jogo e mantém o canal aberto até o servidor terminar a partida com `END WIN`, `END LOSE`, `CANCEL` ou `FULL`.

C. Protocolo textual implementado

O protocolo adotado é simples. Todas as mensagens são trocadas em texto puro e separadas por quebras de linha. As mensagens enviadas pelo servidor são as seguintes:

- `WELCOME <id> <players_total>`: identifica o jogador e informa quantos clientes estão ligados.
- `START <mask> <attempts> <players_total>`: assinala o início da partida e envia o estado inicial.
- `ROUND <round> <active_player_id> <mask> <attempts> <used_letters>`: informa o número da ronda, o jogador ativo e o estado atual.
- `STATE <mask> <attempts> <used_letters>`: difunde o estado do jogo após processamento da jogada.
- `END WIN <winner_id> <word>`: termina a partida com vitória e revela a palavra correta.
- `END LOSE <word>`: termina a partida sem vencedor.
- `CANCEL <reason>`: cancela o lobby quando não existem jogadores suficientes.
- `FULL`: rejeita uma nova ligação por falta de vagas ou porque o jogo já começou.

No sentido cliente para servidor existe uma única mensagem de jogada:

- `GUESS <round> <text>`: envia uma letra ou palavra associada à ronda atual.

O número da ronda faz parte da mensagem `GUESS`. Isso permite ao servidor ignorar jogadas atrasadas ou incoerentes com o turno que está a ser processado. Na prática, o `ClientHandler` só aceita uma mensagem `GUESS` se o número de ronda recebido coincidir com o número esperado.

D. Exemplo de troca de mensagens

O excerto seguinte ilustra uma sequência típica de interação entre dois clientes e o servidor. O exemplo não representa uma captura literal do terminal, mas segue exatamente o formato implementado no protocolo:

```
WELCOME 1 1
WELCOME 2 2
START _______ 6 2
ROUND 1 1 _______ 6 -
GUESS 1 a
STATE _a___a_ 6 a
ROUND 2 2 _a___a_ 6 a
GUESS 2 lagosta
END WIN 2 lagosta
```

Este exemplo mostra que o servidor mantém o controlo do estado, que os clientes apenas reagem às mensagens recebidas e que o fim da partida é indicado por uma mensagem final clara. A Figura 1 apresenta uma captura de ecrã da aplicação em execução, com três clientes e a consola do servidor.

V. Fluxo de Jogo e Gestão do Estado

Depois de o servidor arrancar, é iniciado um temporizador de lobby. Durante este período, os clientes podem ligar-se e recebem a mensagem `WELCOME`. O jogo começa quando entram quatro jogadores ou quando termina o tempo limite do lobby, desde que existam pelo menos dois participantes. Caso contrário, o servidor envia `CANCEL` e encerra a sessão.

Quando a partida se inicia, o `GameManager` escolhe aleatoriamente uma palavra a partir da lista fixa em `WordList`. A palavra é convertida para minúsculas, a máscara é inicializada com sublinhados e o número de tentativas é colocado no valor máximo definido no protocolo.

A cada iteração do ciclo principal, o servidor escolhe um jogador ativo, difunde a mensagem `ROUND` para todos os clientes e aguarda apenas a jogada desse jogador. Depois de receber a mensagem, o método `processGuess()` decide se houve progresso no jogo. Se a jogada for uma palavra correta, a máscara é totalmente revelada e fica definido o vencedor. Se a jogada for apenas uma letra, o servidor percorre a palavra e atualiza todas as posições em que essa letra ocorre.

Caso a jogada não produza qualquer progresso, o servidor consome uma tentativa e envia o novo estado através de `STATE`. Este comportamento abrange letras inexistentes, palavras erradas, entradas vazias por tempo limite e também letras repetidas, uma vez que o servidor interpreta qualquer jogada sem efeito útil como falhada.

VI. Concorrência e Tempos Limite

Sendo uma aplicação com vários clientes ligados em simultâneo, a concorrência tem de ser tratada com cuidado. A solução adotada combina estruturas seguras para acesso concorrente com secções sincronizadas em torno do estado partilhado.

No servidor, a lista de jogadores usa uma `CopyOnWriteArrayList`, o que simplifica a iteração durante difusões e remoções. A atribuição de identificadores é feita através de uma `AtomicInteger`, evitando colisões quando entram novos clientes. Os métodos sensíveis da classe `Server`, como o arranque do jogo, o registo de jogadores prontos e o encerramento do serviço, encontram-se protegidos com `synchronized`. Já a classe `GameManager` protege a atualização da máscara, das tentativas, do conjunto de letras usadas e do vencedor com o mesmo mecanismo.

O tempo limite da jogada é implementado no lado do servidor, no método responsável por recolher a jogada do cliente. Para isso, o `Socket` associado ao jogador ativo recebe

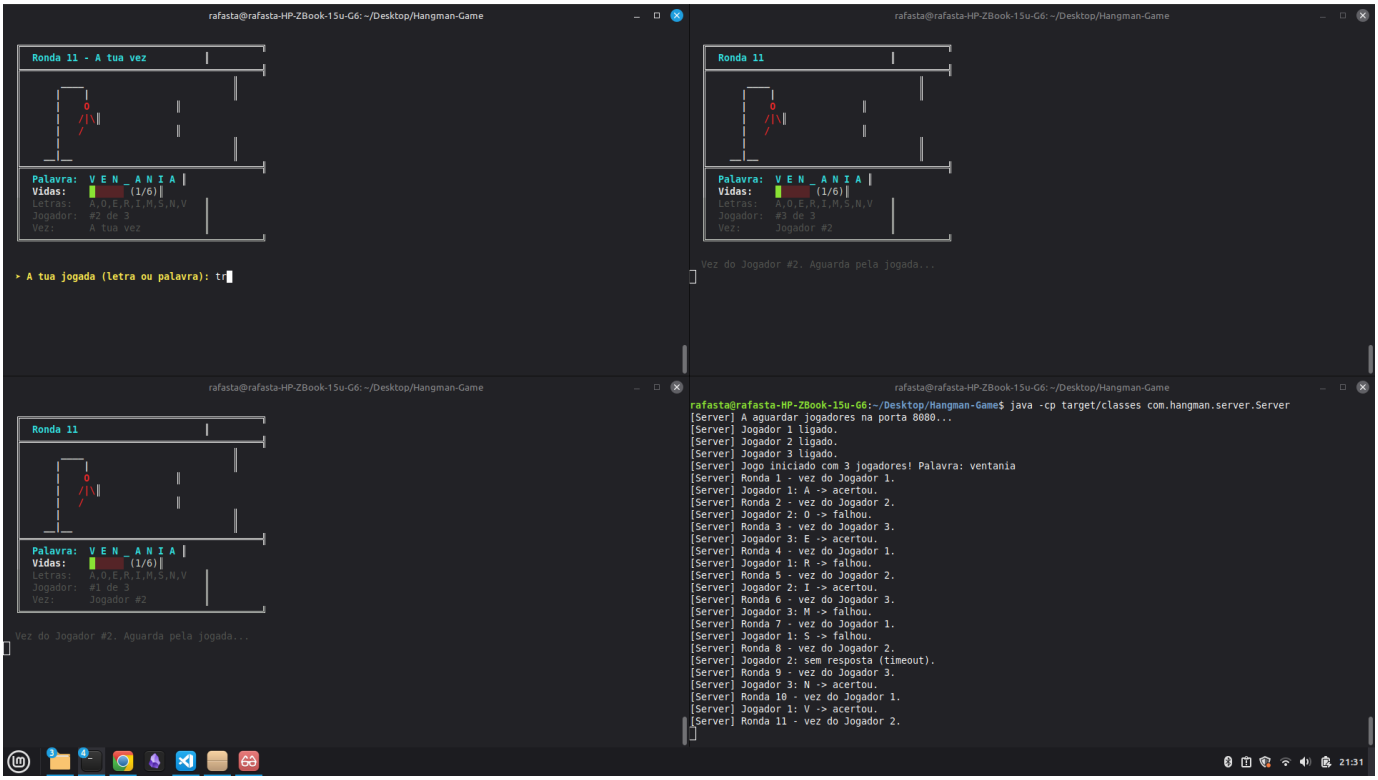

Figura 1. Execução da aplicação com três clientes e o servidor. A imagem mostra a ronda em curso, o estado da palavra, as letras usadas e o registo das jogadas no servidor.

temporariamente um `SoTimeout`. Se o cliente não enviar uma linha válida dentro do intervalo configurado, ocorre uma `SocketTimeoutException` e a jogada é tratada como vazia. Isto evita que o jogo fique bloqueado à espera de um cliente que deixou de responder.

No cliente, a concorrência surge de forma mais simples. Existe uma linha de execução dedicada à receção de mensagens do servidor e outra dedicada à leitura da consola. O estado visível do jogo é guardado em variáveis `volatile`, para que as alterações efetuadas pela receção de mensagens sejam observáveis pela linha responsável pelo envio das jogadas. Além disso, o envio para o servidor é protegido por um bloqueio simples, evitando escrita concorrente sobre o `PrintWriter`.

VII. Decisões de Implementação

O projeto cumpre os aspetos técnicos centrais da proposta: utiliza TCP/IP, suporta múltiplos clientes, centraliza a lógica do jogo no servidor, seleciona uma palavra de uma lista fixa de 100 entradas, trabalha sem distinguir maiúsculas de minúsculas, aplica tempos limite, rejeita novas ligações com `FULL` e termina a sessão com uma mensagem final antes de encerrar os sockets.

Durante o desenvolvimento foram tomadas algumas decisões de implementação. A partida foi organizada em turnos rotativos, existindo um único jogador ativo por ronda. A mensagem `ROUND` inclui o identificador desse jogador, para que todos os clientes saibam de quem é a vez. A mensagem `GUESS` inclui o número da ronda, o que ajuda o servidor a rejeitar jogadas tardias ou desfasadas. A mensagem `START` transporta o estado inicial do jogo e o número de jogadores presentes no momento do arranque.

Estas opções simplificam a coordenação da sessão, reduzem ambiguidades no protocolo e mantêm o controlo do jogo no servidor.

VIII. Compilação e Execução

O projeto usa Java 17 e Maven. A compilação pode ser realizada com:

```
mvn clean compile
```

Para iniciar o servidor:

```
java -cp target/classes com.hangman.server.Server
```

Para iniciar um cliente:

```
java -cp target/classes com.hangman.client.Client
```

Também é possível indicar explicitamente o endereço e o porto do servidor:

```
java -cp target/classes com.hangman.client.Client
```

Durante os testes, basta abrir um terminal para o servidor e dois a quatro terminais adicionais para os clientes. A observação das mensagens trocadas e do comportamento das rondas confirma o funcionamento da ligação cliente–servidor.

IX. Conclusão

O trabalho desenvolvido apresenta uma solução funcional para um jogo da forca distribuído em Java, assente numa

arquitetura cliente–servidor e num protocolo de texto simples sobre TCP/IP. A implementação assenta na centralização do estado do jogo no servidor e na manutenção de ligações persistentes com os vários clientes, o que permite coordenar a sessão de forma consistente.

O projeto mostra competências importantes para a unidade curricular: modelação de protocolos, utilização de sockets, tratamento de concorrência, controlo de tempos limite e encerramento ordenado de sessões distribuídas. No futuro, a aplicação poderá ser alargada com melhorias na interface, registo de partidas ou novos modos de jogo.

REFERÊNCIAS

[1] Oracle, "Class ServerSocket – Java Platform, Standard Edition 17 API Specification," 2021.
[2] Oracle, "Class Socket – Java Platform, Standard Edition 17 API Specification," 2021.
[3] Oracle, "Class Thread – Java Platform, Standard Edition 17 API Specification," 2021.